Network Traffic Anomaly Detector

Technical Design Document

User stories, architecture, data, API and runtime design

Student	**Angel Rusev**
Programme	BSc ICT & Software Engineering
Minor	Cybersecurity — Attack & Defend
Institution	Fontys University of Applied Sciences
Semester	Spring 2026
Document	Technical Design Document v1.0 — companion to the Research Report
Repository	https://github.com/Guts1313/AnomalyDetector

Table of contents

1. Introduction and scope

This Technical Design Document (TDD) is the engineering companion to the Research Report. Where the research document argues why the detector is built the way it is, this document specifies what is built: the user stories the system satisfies, the architecture that realises them, the data it handles, the API it exposes, and the runtime behaviour that ties the pieces together. It is written to be read by an engineer who has to extend, operate, or review the system, and every design statement is traceable to a user story and to code in the accompanying repository.

The scope is the detector itself — a stateless FastAPI inference service, a serialised model bundle, a SQLite audit store, two analyst front-ends (React and Streamlit), and the containerised attack/defend laboratory that drives real offensive traffic through the same inference path. Model research, dataset generation and the per-class evaluation are covered in the Research Report and only summarised here where the design depends on them.

2. Personas and stakeholders

The design serves four personas. The SOC analyst is the primary user of the running system; the ML engineer maintains the model; the NOC operator keeps the service healthy; and the security student or researcher drives the attack/defend laboratory. Table 1 maps each persona to the goals the design must satisfy.

Persona	Goal	Touch-points
SOC analyst	Triage, investigate and explain alerts in real time.	React/Streamlit dashboards, /alerts, /predictions
ML engineer	Retrain, evaluate and hot-reload the model safely.	scripts/train, /admin/reload, /metrics
NOC operator	Keep the service healthy and detect drift.	/health, /metrics, Docker healthchecks
Security student / researcher	Exercise the detector with real attack traffic.	Examples tab, lab attacker/defender containers

Table 1 — Personas and the goals the design must satisfy.

3. User stories

The backlog is organised into six epics, prioritised with MoSCoW and sized in story points. The cards below are the canonical statements; each carries a role, a “so that” benefit,

Given/When/Then acceptance criteria, and links to the functional requirements (FR) and sub-research questions (SRQ) it traces to. The same FR/SRQ tags reappear in the traceability matrix in §13.

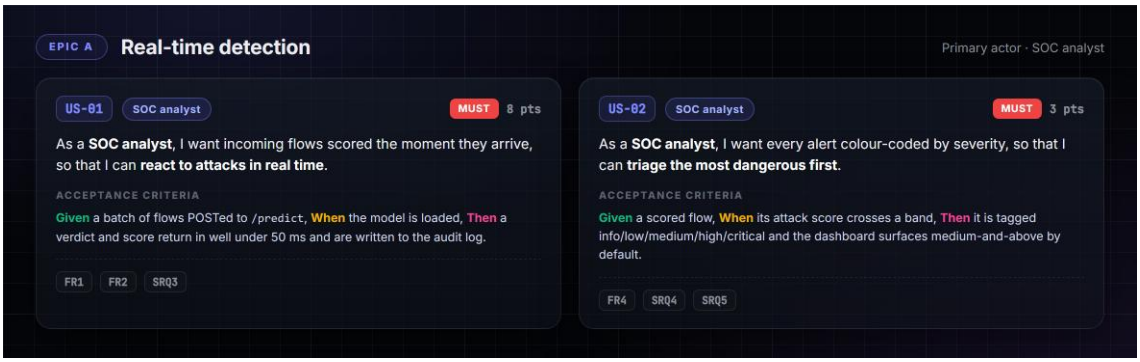

Figure 1 — Epic A — Real-time detection (US-01, US-02).

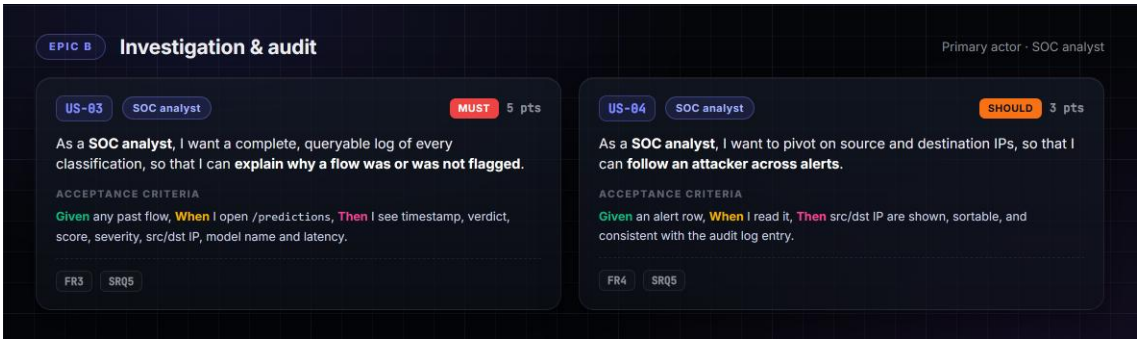

Figure 2 — Epic B — Investigation and audit (US-03, US-04).

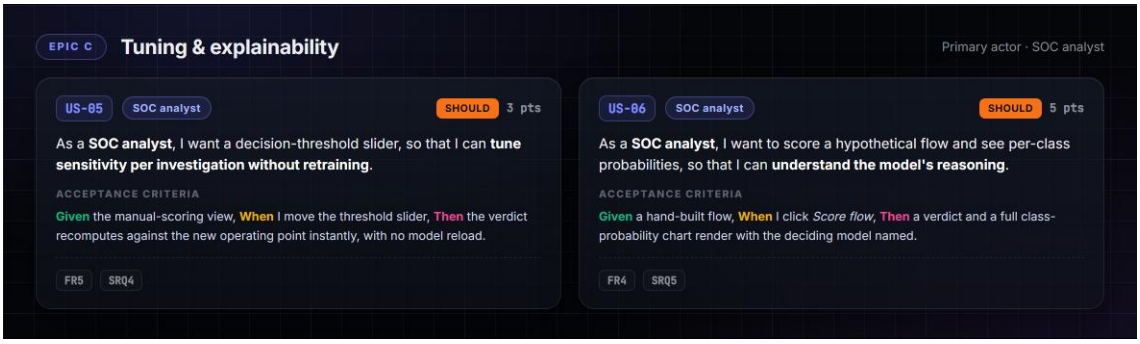

Figure 3 — Epic C — Tuning and explainability (US-05, US-06).

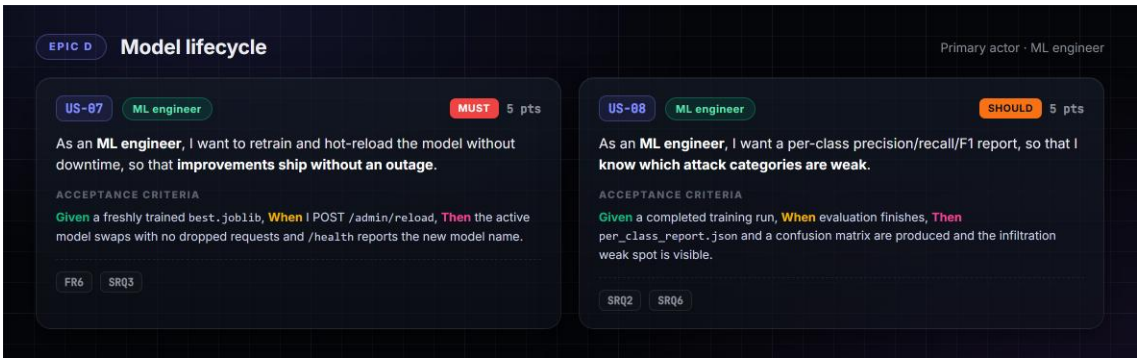

Figure 4 — Epic D — Model lifecycle (US-07, US-08).

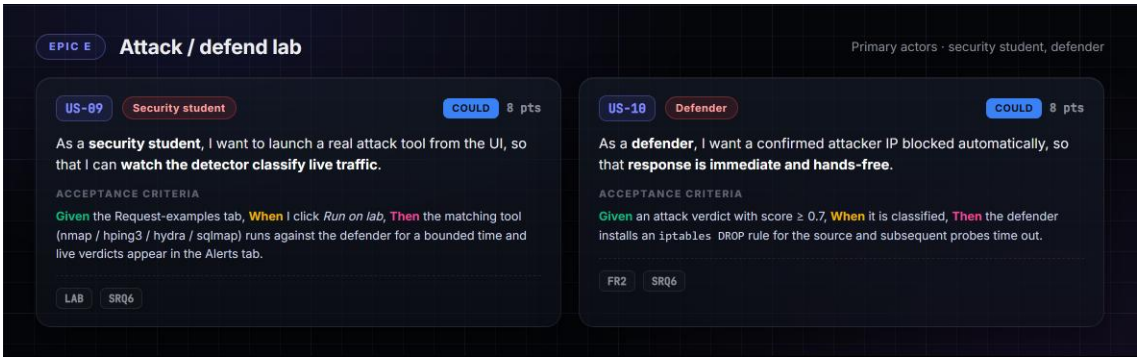

Figure 5 — Epic E — Attack / defend lab (US-09, US-10).

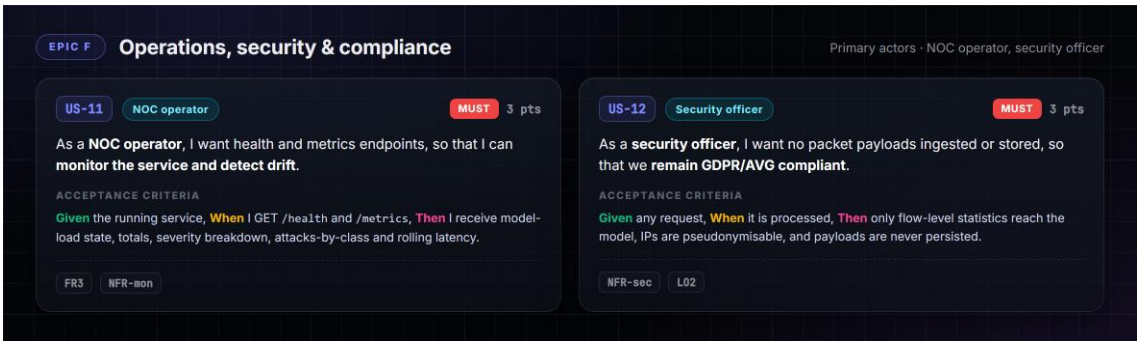

Figure 6 — Epic F — Operations, security and compliance (US-11, US-12).

Table 2 summarises the backlog so it can be read at a glance and sorted by priority. The Must-have stories form the walking skeleton — real-time scoring, severity triage, the audit trail, hot-reload, monitoring and the privacy guarantee — and were delivered first; the Could-have lab stories were the final, highest-value-per-risk increment.

ID	Story (abridged)	Role	Priority	Pts
US-01	Score incoming flows in real time	SOC analyst	Must	8
US-02	Colour-code alerts by severity	SOC analyst	Must	3

US-03	Queryable audit log of every classification	SOC analyst	Must	5
US-04	Pivot on source/destination IPs	SOC analyst	Should	3
US-05	Decision-threshold slider, no retrain	SOC analyst	Should	3
US-06	Manual scoring with class probabilities	SOC analyst	Should	5
US-07	Retrain and hot-reload without downtime	ML engineer	Must	5
US-08	Per-class evaluation report	ML engineer	Should	5
US-09	Launch a real attack from the UI	Security student	Could	8
US-10	Auto-block a confirmed attacker IP	Defender	Could	8
US-11	Health and metrics endpoints	NOC operator	Must	3
US-12	No payloads stored (GDPR)	Security officer	Must	3

Table 2 — Product backlog summary (MoSCoW + story points).

4. Architecture overview

The system is a small set of cooperating containers around one inference service. Figure 7 is the C4 level-1 context: the detector sits between offline flow-feature sources and the human roles it serves, and can forward a severity feed to an upstream SIEM. Figure 8 opens the box one level further into containers.

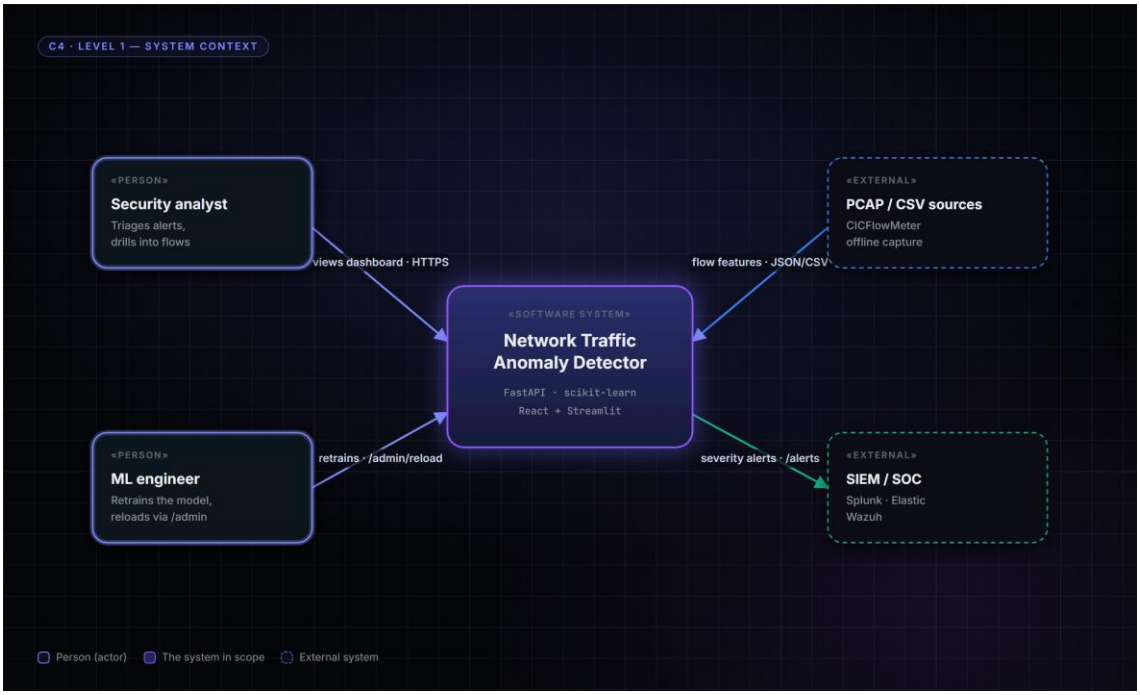

Figure 7 — C4 level-1 context — the detector between flow sources, the analyst, the ML engineer and an optional SIEM.

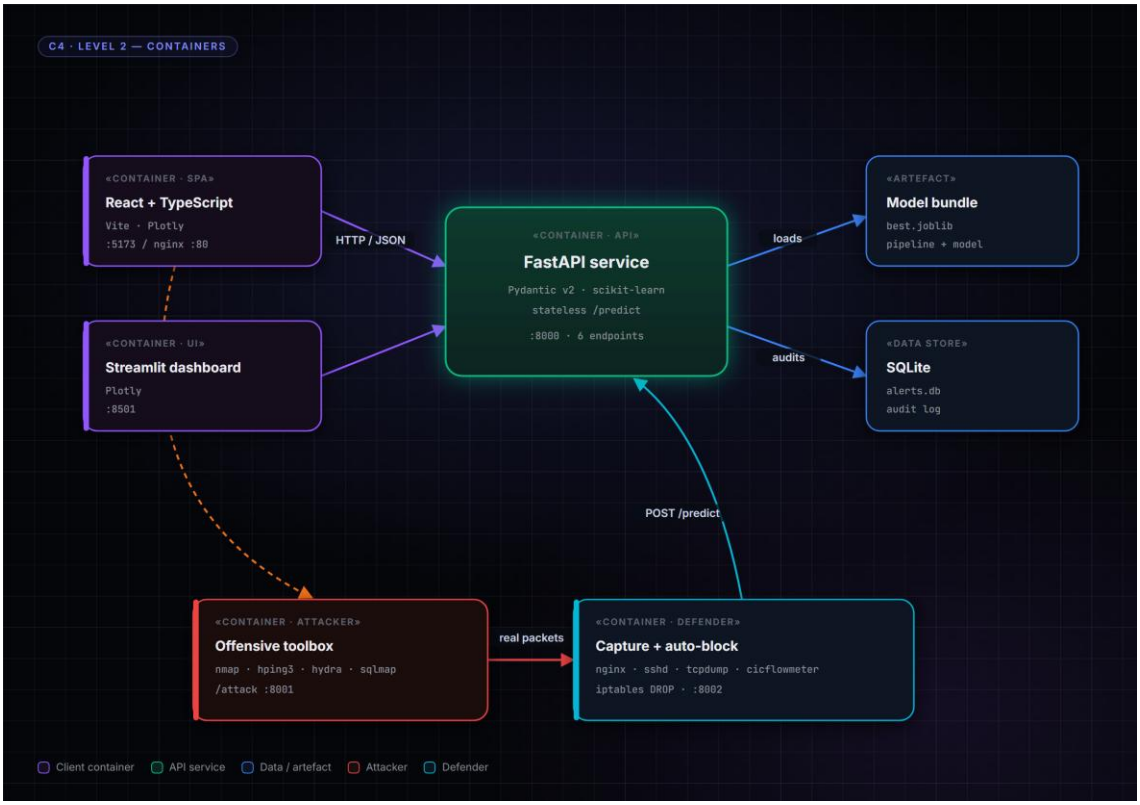

Figure 8 — C4 level-2 containers — the stateless FastAPI hub, its model bundle and SQLite store, the React and Streamlit clients, and the attacker/defender lab.

Three architectural rules are load-bearing and are referenced throughout the design:

- **Stateless inference** — /predict holds no per-request state; the audit write happens after the verdict, off the critical path (US-01).
- **One serialised bundle** — the fitted preprocessing pipeline and the model travel together in best.joblib, so train-time and serve-time transforms cannot diverge (US-07).
- **Batch-first** — /predict always accepts a list of flows so the vectorised scikit-learn path does the work (US-01).

5. Component design

Table 3 lists the runtime components, their responsibility, and the primary user stories each one realises. The boundaries are deliberately sharp: the API never renders UI, the dashboards never touch the model, and the lab containers reach the model only through the same public /predict contract a real client would use.

Component	Responsibility	Realises
FastAPI service (api/main.py)	HTTP surface, request validation, verdict +	US-01, US-02, US-11

	severity, audit write.	
ModelRegistry (models/registry)	Load best.joblib; hot-swap on /admin/reload.	US-07
FeaturePipeline (features/)	Median impute → RobustScaler → one-hot; drop IPs/ports.	US-01, US-12
AlertStore (api/store.py)	SQLite persistence of every classification.	US-03, US-04
React SPA / Streamlit	Severity-coloured dashboards, manual scoring, examples.	US-02, US-05, US-06
Lab attacker (lab/attacker)	Run nmap/hping3/hydra/sqlmap behind /attack.	US-09
Lab defender (lab/defender)	Capture, extract, classify, iptables auto-block.	US-09, US-10

Table 3 — Components, responsibilities and the user stories they realise.

6. Data design

The model consumes a fixed twenty-feature flow vector and nothing else; IPs and ports are carried only as analyst labels and are dropped by the ColumnTransformer before the model sees them — the design-level enforcement of the privacy guarantee in US-12. Table 4 is the feature schema; Figure 9 shows how each attack category expresses those features, which is why the trees can separate them.

Group	Features
Volumetric	flow_duration, total_fwd/bwd_packets, total_length_fwd/bwd_packets, flow_bytes_per_s, flow_packets_per_s
Distributional	fwd/bwd_packet_length_max/mean, flow_iat_mean, flow_iat_std, fwd/bwd_iat_total
TCP flags	fin/syn/rst/psh/ack_flag_count
Protocol	protocol (one-hot)
Labels (dropped)	src_ip, dst_ip, src_port, dst_port

Table 4 — The canonical feature schema (features/schema.py).

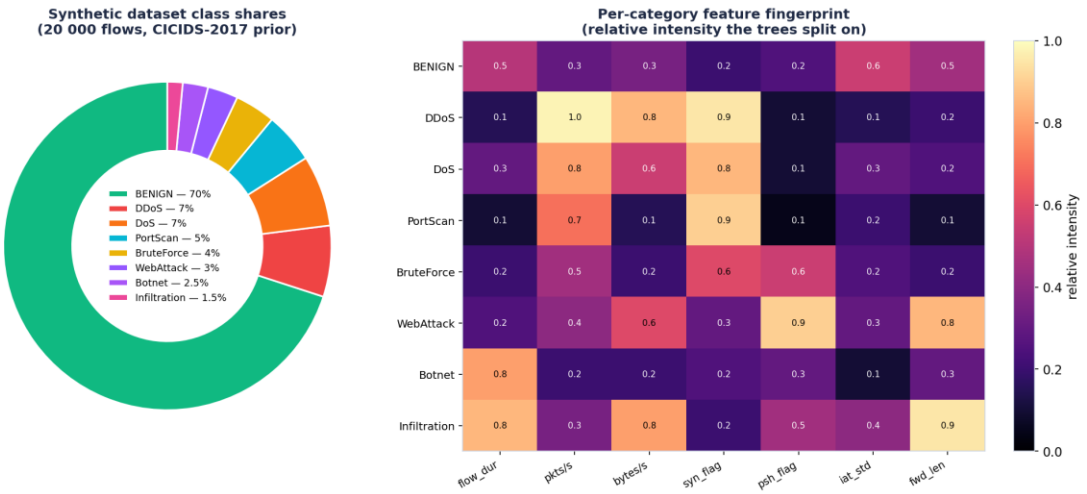

Figure 9 — Dataset class shares and per-category feature fingerprint — the signal the model keys on per attack class.

Persistence is a single SQLite table written after each verdict. Table 5 is its shape; it is what backs the audit and investigation stories (US-03, US-04) and the post-incident replay runbook.

Column	Type	Purpose
id	INTEGER PK	Row identity.
ts	TEXT (ISO)	Classification timestamp (Amsterdam tz).
verdict	TEXT	BENIGN or attack category.
score	REAL	Attack probability.
severity	TEXT	info / low / medium / high / critical.
src_ip / dst_ip	TEXT	Analyst labels (pseudonymisable).
model	TEXT	Deciding model name (traceability).
latency_ms	REAL	Per-request inference latency.

Table 5 — The predictions audit table (api/store.py).

7. API specification

The HTTP surface is intentionally small — six endpoints across four tags — and is fully OpenAPI-documented at /docs. Table 6 is the contract.

Method & path	Tag	Purpose	Story
GET /health	meta	Model-load state + version.	US-11
GET /metrics	meta	Totals, severity, attacks-by-class, latency.	US-11
POST /predict	inference	Score a batch of flows; optional threshold.	US-01, US-05
POST /predict/csv	inference	Score an uploaded CICIDS-format CSV.	US-01
GET /alerts	audit	Severity-filtered alert feed.	US-02, US-04
GET /predictions	audit	Full audit log for triage.	US-03
POST /admin/reload	admin	Hot-reload a new model bundle.	US-07

Table 6 — API surface mapped to user stories.

A representative request and response for the core scoring path (US-01) — a batch of one flow, scored at the default operating point:

```
POST /predict
{ "flows": [ { "protocol": "TCP", "flow_duration": 120000,
               "syn_flag_count": 1, "flow_packets_per_s": 50, ... } ],
  "threshold": 0.5 }

200 OK
{ "results": [ { "verdict": "Infiltration", "score": 0.812,
                 "severity": "high", "model": "gradient_boosting",
                 "probabilities": { "Infiltration": 0.81, "BENIGN": 0.14, ... } } ]
}
```

8. Runtime behaviour

The most instructive runtime path is the attack/defend loop, because it exercises every component end to end. Figure 10 is the sequence: a click on the frontend drives the attacker container, real packets cross the bridge to the defender, the defender extracts a flow and posts it to /predict, the model returns a verdict, and on a confident attack the defender installs an iptables block — while the verdict simultaneously surfaces as a severity-coloured row in the

analyst's Alerts tab. This single diagram realises US-09 and US-10 and demonstrates US-01 through US-03 in passing.

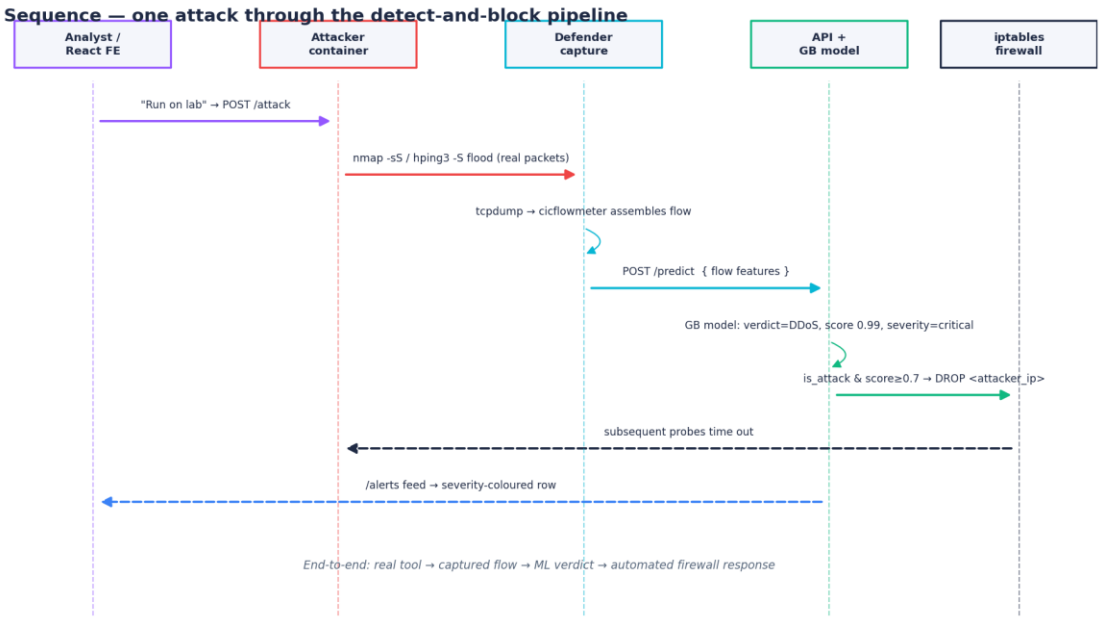

Figure 10 — Sequence — one attack through the detect-and-block pipeline: real tool → captured flow → /predict verdict → automatic iptables block → analyst alert.

The ordinary analyst path is a strict subset of the above: a client posts a batch to /predict, the FeaturePipeline transforms it, the model emits per-class probabilities, the service derives a severity band, writes the audit row asynchronously, and returns the verdict. Because the audit write is off the critical path, a slow disk degrades durability of the log, never the latency of the verdict — the design choice behind US-01's sub-50-millisecond acceptance criterion.

9. Non-functional requirements and security

Table 7 records the non-functional targets and where each is enforced. Security is treated as a design input, not an afterthought: the detector is itself threat-modelled with STRIDE in the Research Report, and the two controls most visible in this design are the no-payload feature schema (US-12) and the non-root, read-only-model container posture.

NFR	Target	Enforced by
Performance	Sub-50 ms verdict per analyst batch	Stateless /predict, batch-first, off-path audit
Monitoring	Every classification observable	/metrics, /health, SQLite audit

Security	No PII in the inference path	schema.py, ColumnTransformer(remainder=drop)
Compliance	GDPR/AVG: pseudonymisable, on-host, revocable	IPs not features; delete alerts.db
Reliability	No-downtime model swap	/admin/reload + ModelRegistry
Usability	Triage at a glance	Severity colour-coding, audit beside queue

Table 7 — Non-functional requirements and their enforcement points.

10. Deployment view

Everything ships as Docker images orchestrated by Compose. The API and dashboard are split into two images so they can scale independently and keep small; both run as dedicated non-root users and mount the model directory read-only. The lab is a separate Compose file with two privileged-capability containers (NET_RAW for capture, NET_ADMIN for iptables) scoped to their own network namespace so the host firewall is never touched — the trust boundary of the offensive stories US-09 and US-10.

- **ad-api** — FastAPI + model bundle, port 8000, non-root user detector.
- **ad-dashboard** — Streamlit, port 8501, non-root user dash.
- **React SPA** — built static assets served by nginx (prod) or Vite (dev), proxying /api.
- **ad-attacker / ad-defender** — the lab, brought up on demand from lab/docker-compose.yml.

11. Testing strategy

Layer	What is tested	Where
Unit — features	Pipeline robustness to NaN/inf, RobustScaler behaviour.	tests/test_features.py
Unit — API	Happy path, attack flow, threshold, 422 on bad input.	tests/ (API surface)
Integration	Train → serve → smoke-test on a small dataset.	CI pipeline
Model eval	Per-class precision/recall/F1, confusion matrix.	scripts/train, per_class_report.json
Lab (manual)	Run-on-lab → verdict → iptables block lands.	lab/ + /blocks endpoint

Table 8 — Test strategy across the layers.

12. Traceability matrix

The matrix closes the loop from need to verification: every user story maps to the component that realises it, the endpoint that exposes it, and the test or evidence that confirms it. It is the single artefact a reviewer can use to check that nothing in the backlog is unbuilt and nothing built is unjustified.

Story	Component	Endpoint / surface	Verified by
US-01	FastAPI + FeaturePipeline	POST /predict	API tests; latency benchmark
US-02	Severity logic + dashboards	/alerts	API tests; dashboard
US-03	AlertStore	GET /predictions	API tests; audit table
US-04	AlertStore + dashboards	/alerts rows	Dashboard review
US-05	Threshold param + slider	POST /predict?threshold	API tests; manual scoring
US-06	Manual scoring view	POST /predict	Frontend; class-prob chart
US-07	ModelRegistry	POST /admin/reload	Reload smoke test
US-08	Training + evaluation	scripts/train	per_class_report.json
US-09	Lab attacker	POST :8001/attack	Manual lab run
US-10	Lab defender	iptables + /blocks	/blocks shows DROP
US-11	Metrics/health	GET /health, /metrics	API tests
US-12	FeaturePipeline schema	(all)	schema.py; no payload stored

Table 9 — Traceability: story → component → endpoint → verification.

References

[1] Research Report — Network Traffic Anomaly Detector (companion document).

[2] C4 model for software architecture. https://c4model.com/

[3] Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward Generating a New Intrusion Detection Dataset. ICISSP.

[4] FastAPI documentation. https://fastapi.tiangolo.com/

[5] scikit-learn: Novelty and Outlier Detection. https://scikit-learn.org/stable/modules/outlier_detection.html

[6] OWASP API Security Top 10 (2023). https://owasp.org/API-Security/